

Arquitetura Concorrente em C++20 para Geração Procedural e Renderização de Malhas 3D

André Vitor Bastos de Macêdo
Instituto Federal de Educação, Ciência e Tecnologia Catarinense
Campus Blumenau
andre.macedo@estudantes.ifc.edu.br

Ricardo de la Rocha Ladeira
Instituto Federal de Educação, Ciência e Tecnologia Catarinense
Campus Blumenau
ricardo.ladeira@ifc.edu.br

RESUMO

A geração procedural e a extração geométrica de malhas tridimensionais densas impõem desafios arquiteturais críticos, frequentemente sobrecarregando a *thread* principal e degradando a fluidez da renderização gráfica. Este artigo apresenta uma arquitetura assíncrona baseada no padrão *Task Scheduler*, implementada em C++20, para gerenciar a geração e preparação concorrente de terrenos baseados em Ruído de Perlin. O estudo demonstra como o isolamento do contexto OpenGL e o uso de filas de prioridade multinível otimizam a aquisição de dados e mantêm a estabilidade do *framerate* em simulações intensivas.

CCS Concepts:

- **Computing methodologies** → Rendering
- **Computing methodologies** → Concurrent computing methodologies

Keywords: Procedural Generation, C++20, Graphics Engine, Concurrent Programming, OpenGL

1 INTRODUÇÃO

Gerar terrenos de forma procedural e preparar malhas 3D complexas são tarefas que exigem muito processamento. Quando essas operações são feitas ao mesmo tempo em que o motor gráfico tenta desenhar na tela, a *thread* principal fica sobrecarregada. Isso faz com que a taxa de quadros (FPS) caia drasticamente, causando travamentos na aplicação.

Motores gráficos que usam OpenGL sofrem ainda mais com esse problema. Por design, o OpenGL exige que o laço de desenho e as alterações de estado dos gráficos aconteçam exclusivamente na *main thread*. Se essa mesma *thread* tiver que parar para calcular um mapa de ruído ou gerar a geometria da malha, a renderização é interrompida. Portanto, é preciso isolar o processamento pesado para garantir que a interface continue responsiva.

Para resolver isso, este artigo apresenta a implementação de um *Task Scheduler* [10] desenvolvido com os recursos modernos do C++20. O sistema utiliza filas de prioridade multinível para organizar a criação dos mapas de altura via Ruído de Perlin e a extração das malhas em *threads* de *background* (trabalhadoras), deixando a *main thread* livre apenas para as chamadas gráficas.

A contribuição deste trabalho é demonstrar como essa arquitetura consegue separar a renderização da preparação dos dados de forma eficiente. Além disso, mostra-se como o uso de `std::jthread` e *smart pointers* facilita o gerenciamento de memória em sistemas paralelos, evitando erros críticos e garantindo que o motor gráfico continue rodando de forma fluida mesmo durante simulações intensas.

2 FUNDAMENTAÇÃO TEÓRICA

Este estudo se baseia em conceitos fundamentais de computação gráfica, geração procedural e técnicas de concorrência em C++. A

seguir, serão discutidos os principais tópicos relacionados a esses conceitos.

2.1 Geração Procedural de Terrenos

A criação de ambientes de teste diversificados fundamenta-se na geração procedural de mapas de altura (*heightmaps*). Para este estudo, utiliza-se o Ruído de Perlin [8], um algoritmo de ruído gradiente que gera padrões pseudoaleatórios com transições suaves, simulando topografias naturais. A complexidade do relevo é obtida pela sobreposição de múltiplas camadas de ruído, denominadas oitavas (*octaves*).

O controle do detalhamento é exercido por dois parâmetros: a lacunaridade, que regula o aumento da frequência entre oitavas sucessivas, e a persistência, que controla a redução da amplitude. O mapa de ruído resultante é mapeado para uma grade tridimensional, onde a intensidade de cada pixel define a elevação do vértice correspondente. Essa malha serve como base geométrica para os testes de performance da arquitetura proposta.

2.2 Renderização e Responsividade

Este estudo fará uso da OpenGL API [9] para a renderização das malhas 3D. O loop de renderização típico envolve a atualização do estado da aplicação, o processamento de eventos e a renderização dos gráficos. A OpenGL é sensível a bloqueios na *main thread*, o que significa que se a *main thread* estiver ocupada processando a geometria das malhas, a renderização pode ser interrompida, resultando em travamentos ou quedas de desempenho.

Para a implementação do motor gráfico será utilizada a biblioteca GLFW, que fornece uma interface de criação de janelas, contextos OpenGL e gerenciamento de eventos de entrada. O loop de eventos do GLFW é projetado para trabalhar como uma máquina de estado, onde a *main thread* é responsável por processar eventos e renderizar gráficos. Se a *main thread* estiver bloqueada por tarefas de processamento intensivo, como a geração de terrenos procedurais, o loop de eventos e de desenho ficará bloqueado, o que compromete a responsividade geral do sistema.

Se a *main thread* assumir o custo computacional da geração e preparação das malhas 3D, o loop de desenho sofrerá quedas de *framerate* (FPS) e engasgos (*stuttering*). Com isso, é essencial isolar essas tarefas para garantir que a renderização do OpenGL não seja interrompida.

2.3 Monitor

Conforme explica Ricardo de la Rocha Ladeira. [3], o padrão Monitor é uma implementação de alto nível para controle de sincronização em programação paralela. Ele utiliza mecanismos de bloqueio (*mutexes*) para garantir exclusão mútua e variáveis de condição para coordenar a execução das *threads*, prevenindo condições de corrida (*race conditions*) entre as rotinas assíncronas de geração de dados e a *main thread*. Ao encapsular o estado compartilhado, o monitor ajuda a garantir que o processamento das malhas possa ser realizado em paralelo de forma eficiente.

2.4 Agendamento de Tarefas (*Task Scheduler*)

Um *Task Scheduler* é um componente de software responsável por gerenciar a execução de tarefas concorrentes. Ele mantém uma fila de tarefas pendentes e um conjunto de threads (*thread pool*) que processam essas tarefas em paralelo [4]. O *Task Scheduler* é projetado para otimizar o uso dos recursos do sistema, garantindo que as tarefas pesadas de geração de terrenos sejam delegadas para threads trabalhadoras (*worker threads*), mantendo a *main thread* sempre responsiva para a renderização gráfica.

2.4.1 Filas multinível

O *Task Scheduler* implementará filas multinível, onde as tarefas são categorizadas por prioridade. As tarefas de alta prioridade incluem a geração das configurações iniciais e mapas base, enquanto as tarefas de média prioridade envolvem a construção da malha 3D. Tarefas de baixa prioridade são relacionadas à exportação de dados ou logs que não impactam diretamente a fluidez da aplicação.

2.5 Concorrência Moderna com C++20

2.5.1 Gerenciamento de Threads

A linguagem C++20 introduziu várias melhorias para a programação concorrente, incluindo a classe `std::jthread`, que é uma extensão da classe `std::thread` com suporte integrado para cancelamento de threads. O uso de `std::jthread` permite que as threads sejam encerradas de forma limpa e segura, evitando problemas comuns como deadlocks e condições de corrida.

Essa classe utiliza de métodos RAII (*Resource Aquisition is Initialization*), que é uma técnica de gerenciamento de recursos que garante que os recursos sejam adquiridos e liberados de forma segura, mesmo em casos de exceção. Com `std::jthread`, as threads são automaticamente unidas (*joined*) em chamadas de destruição. A união de threads é o processo onde a *thread* principal espera que a thread trabalhadora termine sua execução antes de continuar, garantindo que os recursos sejam liberados adequadamente.

Ou seja, quando um objeto `std::jthread` é destruído, ele automaticamente chama `join()` na thread associada, garantindo que a thread seja concluída antes de liberar os recursos. Isso garante que as threads sejam encerradas de forma segura, evitando problemas como threads zumbis ou recursos não liberados.

Em seu construtor, as `std::jthread` recebem uma função lambda que encapsula a lógica de execução da thread trabalhadora. Funções lambda são funções anônimas que podem capturar variáveis do escopo onde foram definidas, o que facilita a passagem de dados para as threads trabalhadoras. Quando uma `std::jthread` é instanciada, ela inicia a execução da função lambda em uma nova thread.

2.5.2 Tokens de Parada

O `std::stop_token` é um mecanismo que permite sinalizar a uma thread que ela deve parar sua execução. Isso é especialmente útil para garantir que as threads possam ser encerradas de forma cooperativa, evitando bloqueios. Quando uma `std::jthread` é instanciada sabemos que ela recebe uma função lambda, porém caso essa função lambda tenha um *loop* interno e precise ser interrompida antes de sua conclusão, o `std::stop_token` pode ser utilizado para sinalizar a thread para que ela pare sua execução de forma segura.

No seu loop, a função lambda pode verificar periodicamente o estado do `std::stop_token` para determinar se deve continuar executando ou encerrar a thread. Isso permite que as threads sejam encerradas de forma cooperativa, evitando bloqueios e garantindo que os recursos sejam liberados adequadamente.

Naturalmente, a função deve estar projetada para receber um `std::stop_token` como argumento para que o loop interno possa verificar o seu estado. Outra vantagem do uso de `std::jthread` é que ele integra uma injeção automática de um `std::stop_token` para a função lambda caso não seja passado explicitamente e ele detecte que a função lambda tem um parâmetro do tipo `std::stop_token`. Ou seja, não é necessário instanciar e passar explicitamente um `std::stop_token` para a função lambda, pois ele é automaticamente injetado pela `std::jthread`.

2.5.3 Variáveis Condicionais

O `std::condition_variable_any` é uma ferramenta de sincronização que permite que as threads esperem por condições específicas, facilitando a coordenação entre as threads trabalhadoras e a *main thread*. Ele é utilizado para implementar a lógica de espera e notificação entre as threads. Por exemplo, quando as threads trabalhadoras são criadas, todas são colocadas em espera usando `std::condition_variable_any`, aguardando que a *main thread* adicione uma tarefa na fila. Isso sinaliza a `std::condition_variable_any` para que acorde uma das threads trabalhadoras para processar a tarefa.

As threads trabalhadoras, por sua vez, também podem notificar a *main thread* quando uma tarefa é concluída, permitindo que ela atualize o estado do sistema ou inicie novas tarefas conforme necessário. O uso de `std::condition_variable_any` é crucial para garantir que as threads possam coordenar suas ações de forma eficiente, evitando bloqueios e garantindo que as tarefas sejam processadas em ordem.

2.6 Segurança de Memória e Compartilhamento de Estado

O uso de ponteiros brutos do C++ em um ambiente de programação concorrente pode levar a problemas de segurança de memória gravíssimos, como condições de corrida e falhas de segmentação (SIGSEGV). Uma vez que as threads trabalhadoras operam sobre os dados compartilhados, é crucial garantir que esses dados permaneçam válidos durante toda a execução das tarefas.

Para isso, a implementação do *TaskMaster* banirá o uso de ponteiros brutos e adotará exclusivamente referências e smart pointers (como `std::shared_ptr` ou `std::weak_ptr`) para gerenciar o acesso aos dados compartilhados.

Referências são uma forma segura de acessar um objeto em memória sem a necessidade de lidar com a alocação e desalocação manual de memória, o que reduz significativamente o risco de erros de memória. Diferente de cópias, as referências não criam uma nova instância do objeto, mas sim apontam para o mesmo objeto em memória. Em contradição à ponteiros brutos, as referências não podem ser nulas, o que elimina a possibilidade de acessar um ponteiro nulo e causar uma falha de segmentação.

Smart pointers, por outro lado, são objetos que gerenciam automaticamente a vida útil de um recurso, como um objeto alocado dinamicamente. Um `std::shared_ptr` permite que múltiplos objetos compartilhem a propriedade de um recurso, garantindo que ele seja destruído apenas quando a última referência for liberada. Enquanto um `std::weak_ptr` garante que apenas um objeto tenha a propriedade de um recurso.

Ao utilizar referências e smart pointers, o *TaskMaster* garante que os valores de ruído e as estruturas de malha compartilhadas entre as threads trabalhadoras permaneçam válidos durante toda a execução das tarefas, garantindo a integridade dos dados.

3 METODOLOGIA

Para avaliar o impacto do processamento paralelo na geração das malhas, desenvolveu-se uma arquitetura baseada no padrão *Task Scheduler*. A metodologia adotada divide-se na implementação estrutural do escalonador, na instrumentação para coleta de dados de desempenho e na definição de cenários de teste isolados.

3.1 Ambiente de Teste

O hardware utilizado para os testes consiste em um processador de 12ª geração Intel Core i5-1235U (12 *threads*, com frequência máxima de 4.40 GHz) e 16 GB de memória RAM. A aceleração gráfica foi provida por uma GPU integrada Intel Iris Xe Graphics.

O sistema operacional utilizado foi Arch Linux (Kernel 6.15.9). Todo o código-fonte foi desenvolvido obedecendo estritamente ao padrão C++20 e compilado utilizando a coleção de compiladores GNU (GCC). Para avaliar a máxima performance dos algoritmos, o binário final foi gerado utilizando a flag de otimização de tempo de execução -O3 (Release), juntamente com diretrizes rigorosas de compilação (-Wall, -Wextra, -Wpedantic e -Werror).

A renderização e o controle do *loop* de eventos foram construídos utilizando a API nativa do OpenGL versão 4.6 em conjunto com a biblioteca de gerenciamento de janelas GLFW versão 3.4. As operações matemáticas em matrizes e vetores tridimensionais foram realizadas através da biblioteca GLM (*OpenGL Mathematics*) versão 1.0.3. O gerenciamento de dependências e a orquestração do *build* foram realizados por meio da ferramenta CMake.

3.2 Cenários de Teste

Para garantir a aquisição de dados que reflitam o comportamento real dos algoritmos, os experimentos foram estruturados em quatro modos de execução distintos, cruzando o isolamento do processamento com a concorrência:

- **Bench Sequential (BS):** Geração e extração linear em ambiente isolado (sem motor gráfico), servindo como *baseline* de performance pura.
- **Bench Parallel (BP):** Geração e extração utilizando o TaskMaster para processamento paralelo, medindo o escalonamento da CPU sem interferência da GPU.
- **Engine Sequential (ES):** Integração com o motor gráfico onde a geração ocorre na *main thread*, bloqueando o laço de renderização e permitindo medir a degradação do FPS.
- **Engine Parallel (EP):** Geração assíncrona em *threads* de *background* com upload de dados para o motor conforme a disponibilidade, validando a fluidez da aplicação.

Os testes foram realizados variando-se dois parâmetros fundamentais do relevo: a **Escala** (densidade da malha de 100x100 até 1000x1000 vértices) e o **Número de Oitavas** (complexidade geométrica de 1 a 10 camadas de ruído). Para cada nível de complexidade, foram coletadas 20 amostras, garantindo robustez estatística aos resultados.

3.3 Instrumentação e Coleta de Dados

Para coletar e organizar os resultados de desempenho, foi desenvolvido um *wrapper* que estrutura os dados de forma hierárquica. O objetivo principal deste componente é facilitar a exportação e análise das métricas coletadas, mantendo o código de teste organizado e os resultados fáceis de interpretar.

A estrutura utilizada para o armazenamento em memória é `std::map<std::string, std::map<std::string, std::vector<double>>>`. Nela, o primeiro nível associa a configuração de geração (como “Escala” ou “Número de oitavas”) a um conjunto de métricas. O segundo nível vincula cada métrica (como “Tempo de Extração” ou “FPS Médio”) a um vetor que armazena os valores obtidos em cada repetição do experimento.

Embora o uso de `std::map` envolva mais alocações dinâmicas do que um vetor contíguo, essa escolha não interfere na precisão dos resultados. Isso ocorre porque o registro dos dados no módulo de instrumentação é feito apenas após a finalização da medição de dados de cada algoritmo. Assim, a abordagem prioriza a facilidade em adicionar novas métricas sem comprometer o desempenho medido nos benchmarks.

3.3.1 Dados de Desempenho Coletados

Para analisar o impacto dos algoritmos, foram selecionadas métricas que avaliam tanto a eficiência computacional quanto a qualidade da solução encontrada:

- **Tempo de Extração:** Mede o intervalo total gasto para converter o mapa de ruído em uma malha de triângulos, sendo uma métrica crítica para a fluidez do sistema.
- **Eficiência do Cache:** Avalia a taxa de acertos e falhas no cache da CPU via ferramenta `perf stat` do Linux, fornecendo insights sobre a localidade de referência dos algoritmos e seu impacto no desempenho.
- **Responsividade (FPS):** A taxa de quadros por segundo da aplicação é monitorada para observar como a geração assíncrona da malha permite manter a fluidez da renderização na thread principal.

Além dessas métricas, o sistema permite registrar parâmetros de configuração do ruído, facilitando a correlação entre a complexidade da malha e o custo de renderização. As possíveis variações de configuração incluem:

- **Escala:** Refere-se à densidade da malha, que influencia o número de vértices gerados e o tempo de transferência para a GPU.
- **Número de oitavas:** Determina a quantidade de camadas de ruído sobrepostas, afetando a complexidade do terreno e, consequentemente, o desempenho da geração de mapas.

3.4 Pipeline de Processamento Sequencial

Para garantir a validade estatística dos resultados, o pipeline sequencial (modos BS e ES) atua como o modelo de referência. Neste fluxo, todas as etapas de geração de ruído e triangulação ocorrem de forma linear na thread principal.

O fluxo de processamento segue uma ordem rígida: para cada configuração de teste, o sistema gera o mapa de ruído e realiza a extração dos vértices antes de prosseguir para a próxima amostra ou quadro de renderização. As baterias de testes foram organizadas em 8 níveis incrementais de complexidade (passos), variando a escala de 100x100 a 800x800 ou o número de oitavas de 1 a 10.

Embora funcional para volumes menores de dados, esta abordagem sequencial revela limitações críticas conforme a complexidade aumenta. No modo ES, o custo acumulado da geração de relevo na mesma thread de renderização causa quedas bruscas de FPS e travamentos visíveis, o que motiva e justifica a transição para a arquitetura concorrente (modos BP e EP).

3.5 Arquitetura da Solução Concorrente

Com o objetivo de isolar a thread principal e permitir que as tarefas de extração e construção de dados sejam processadas em paralelo, foi implementada uma arquitetura baseada no padrão *Task Scheduler*. Esta arquitetura é projetada para gerenciar o fluxo de tarefas concorrentes, garantindo que a *main thread* permaneça responsiva enquanto as tarefas de extração e construção de dados são processadas em paralelo.

Para isso, foi criada uma classe TaskMaster que encapsula a lógica de gerenciamento de tarefas e threads. O TaskMaster é responsável por manter uma fila de tarefas pendentes, gerenciar

um pool de threads trabalhadoras e coordenar a execução das tarefas de forma eficiente.

3.5.1 Estrutura do TaskMaster

Esse componente é projetado para ser o núcleo do sistema de agendamento de tarefas, gerenciando a execução concorrente das tarefas de extração e construção de dados. Ele mantém uma fila de tarefas pendentes e um conjunto de threads trabalhadoras que processam essas tarefas em paralelo.



Figura 1: Diagrama de classes da arquitetura do TaskMaster

A implementação do TaskMaster utiliza um conjunto de três filas de prioridade, representadas pelo enum class Priority com os níveis High (0), Medium (1) e Low (2). Essas filas são armazenadas em um `std::array` de `std::queue`, permitindo um acesso eficiente e organizado por nível de importância.

No construtor da classe, o número de threads trabalhadoras é determinado dinamicamente através de `std::thread::hardware_concurrency()`. Para evitar a saturação completa dos núcleos do processador e garantir que a *main thread* (responsável pela renderização e interface) permaneça responsiva, o sistema reserva um núcleo, instanciando $N - 1$ threads trabalhadoras. Estas threads são implementadas como objetos `std::jthread`, aproveitando o comportamento RAII para garantir que sejam finalizadas corretamente na destruição do escalonador.

Cada thread trabalhadora executa um loop contínuo que aguarda por novas tarefas utilizando uma `std::condition_variable_any`. A lógica de seleção de tarefas prioriza sempre as filas de maior importância: o trabalhador verifica sequencialmente as filas High, Medium e Low, extraindo a primeira tarefa disponível na fila de maior prioridade encontrada. Isso garante que tarefas críticas, como a geração da malha visível, sejam processadas antes de tarefas de menor impacto, como a exportação de dados estatísticos.

O loop de execução é projetado para ser interrompido de forma cooperativa através do uso de `std::stop_token`, permitindo que as threads sejam encerradas de forma segura quando o escalonador for destruído ou quando uma interrupção for solicitada.

```

workers.emplace_back([this, id](std::stop_token st) {
    while (!st.stop_requested()) {
        // Lógica de espera e execução de tarefas
    }
});

```

Em cada iteração, a thread trabalhadora aguarda por uma notificação indicando que uma nova tarefa foi adicionada à fila. A espera é implementada utilizando `std::condition_variable_any`, que bloqueia a thread até que uma tarefa esteja disponível ou até que uma solicitação de parada seja feita.

```

// Bloco de código para uso do lock
{
    std::unique_lock<std::mutex> lock(mtx);
    auto stopCon = [this]{
        return

```

```

!taskQueues[0].empty() ||
!taskQueues[1].empty() ||
!taskQueues[2].empty();
};
if (!cv.wait(lock, st, stopCon)) return;

```

Após ser notificada, a thread trabalhadora verifica as filas de tarefas em ordem de prioridade. A primeira tarefa disponível na fila de maior prioridade é extraída e executada. Se nenhuma tarefa estiver disponível, a thread retorna ao estado de espera. Mesmo que um lock tenha sido adquirido, a função `wait` do `std::condition_variable_any` é projetada para liberar o lock enquanto a thread está bloqueada, permitindo que outras threads adicionem tarefas à fila. Quando a thread é acordada, o lock é automaticamente re-adquirido e destruído com o escopo do bloco.

```

for (int q = 0; q < 3; ++q) {
    if (!taskQueues[q].empty()) {
        task = std::move(taskQueues[q].front());
        taskQueues[q].pop();
        break;
    }
}
// Fim do bloco de código para uso do lock
}

```

```

if (task) task(st);

```

O método `addTask` é o ponto de entrada para a submissão de tarefas. Utilizando modelos de programação (*templates*) e metaprogramação em tempo de compilação (`if constexpr`), o método é capaz de aceitar funções que recebem ou não um `std::stop_token`. Isso confere flexibilidade ao sistema, permitindo que tarefas de longa duração verifiquem periodicamente se uma interrupção foi solicitada, enquanto tarefas curtas e simples podem ser executadas sem essa complexidade adicional. Na Figura 2, observa-se como o método `addTask` emprega metaprogramação para unificar a submissão de tarefas, além de notificar a variável de condição para despertar uma thread ociosa.

O uso de metaprogramação na passagem da função lambda permite que o TaskMaster possa lidar com uma variedade de tarefas sem exigir que todas as funções de tarefa sejam projetadas para aceitar um `std::stop_token`.

Encapsulando a tarefa, é mais fácil manter a fila de prioridade sem abstrações desnecessárias. Então, o método `addTask` verifica se a função fornecida é invocável com um `std::stop_token`. Se for, a função é usada diretamente. Caso contrário, ela é encapsulada em uma função lambda que ignora o `std::stop_token`, permitindo que tarefas simples sejam adicionadas sem a necessidade de lidar com tokens de parada.

Com a função encapsulada, o método `addTask` adquire um lock para garantir acesso exclusivo às filas de tarefas e adiciona a tarefa à fila correspondente à sua prioridade. Após a adição, a variável de condição é notificada para acordar uma thread trabalhadora que esteja aguardando por novas tarefas, garantindo que a tarefa seja processada o mais rápido possível.

A sincronização entre as threads trabalhadoras e a thread principal é garantida por um `std::mutex`, protegendo o acesso às filas de tarefas e à variável de condição. O uso de `std::condition_variable_any` permite que as threads trabalhadoras sejam notificadas imediatamente quando uma nova tarefa é adicionada, evitando a necessidade de polling e garantindo uma resposta rápida às mudanças na fila de tarefas.

Por fim, o ciclo de vida do escalonador é encerrado de forma cooperativa através do seu destrutor (Listagem 3). O uso combinado

de `notify_all` para acordar threads bloqueadas e `request_stop` do `std::jthread` garante um encerramento limpo e livre de *dead-locks*.

3.5.2 Paralelização da Aquisição de Dados

A preparação da geometria e a extração de métricas de desempenho foram fortemente beneficiadas pela arquitetura do TaskMaster e pelo uso de *multithreading* direto. O processo compreende a geração procedural do terreno (mapa de ruído), a extração da malha tridimensional correspondente e o upload dos dados para a GPU.

No modo EP (Engine Parallel), utiliza-se uma thread de *background* dedicada que produz continuamente novos dados de malha e os insere em uma fila segura (`std::queue` protegida por `std::mutex`). O laço principal da engine consome as malhas prontas da fila de forma assíncrona. Isso permite que a renderização ocorra de forma fluida enquanto novos setores do terreno são processados em paralelo, eliminando o bloqueio da *main thread*.

Para o modo BP (Bench Parallel), o TaskMaster distribui as repetições do experimento entre múltiplos núcleos, utilizando mecanismos de sincronização (`std::mutex` e `std::condition_variable`) para orquestrar o fim de cada passo de teste. Dessa forma, a *main thread* aguarda a conclusão de todos os trabalhadores antes de consolidar as estatísticas através da classe *Statistics*.

3.6 Controle de Recursos do Sistema Operacional

Para garantir um ambiente isolado e obter um sistema quiescente, livre de ruído experimental [2], o sistema operacional foi configurado para reduzir a influência de processos em segundo plano. Um script de inicialização foi criado para desativar serviços desnecessários e ajustar a política de escalonamento da CPU e configurar o sistema para operar em modo *performance*.

Além disso, todos os testes de benchmark (BS e BP) foram conduzidos no ambiente TTY (sem interface gráfica), garantindo que a GPU não fosse utilizada para renderização de janelas ou efeitos visuais do sistema operacional durante os testes. Essa abordagem assegura que os resultados obtidos reflitam com precisão o desempenho da arquitetura concorrente implementada, conforme preconizado pelas boas práticas de medição de sistemas computacionais [5].

Após os ajustes no sistema, os testes são executados 5 vezes alternativamente, ou seja um após o outro, com 10 segundos de espera entre cada execução para garantir que o sistema esteja estabilizado e que os resultados sejam consistentes.

4 RESULTADOS

O desempenho dos algoritmos foi avaliado com base nas métricas de tempo de extração, eficiência do cache e responsividade (FPS). Os resultados obtidos demonstram uma melhoria significativa no desempenho quando a arquitetura concorrente é utilizada, especialmente em cenários de alta complexidade.

4.1 Tratamento de dados

Mesmo rodando o algoritmo de forma isolada em linha de comando, interrupções temporárias do sistema operacional, oscilações na frequência da CPU (thermal throttling) ou pequenos atrasos de alocação de memória podem gerar picos isolados de latência (outliers) [7]. Para tratar esses pontos de dados discrepantes de forma matematicamente rigorosa (como recomendado para relatórios acadêmicos), implementamos o método da Amplitude Interquartilica (IQR) [1]:

- **Deteção:** Para cada configuração de teste (mesma escala/oitava e mesmo modo), calculou-se a amplitude interquartilica. Valores de tempo fora do intervalo das amostras foram classificados como outliers.
- **Imputação de valores:** Os outliers detectados foram substituídos pela **mediana** do seu respectivo grupo. Isso preserva o tamanho amostral original ($N = 100$) e a tendência central, mas estabiliza o desvio padrão e o erro residual, garantindo que o modelo da ANOVA atenda ao pressuposto de homogeneidade de variância.

4.2 Comparação de benchmark

Para os testes de benchmark, os modos BS e BP foram comparados para avaliar o impacto do processamento paralelo na geração das malhas. Os dados foram analisados utilizando ANOVA de dois fatores com nível de significância $\alpha = 0,05$, considerando a Escala e o Número de Oitavas como fatores independentes [6]. A análise relevou diferenças estatisticamente significativas entre os modos de execução, com o modo BP apresentando tempos de extração consistentemente menores em todas as configurações testadas. A análise estatística foi conduzida utilizando ANOVA de dois fatores, com nível de significância $\alpha = 0,05$. O teste de Tukey foi aplicado para identificar diferenças significativas entre os grupos [6].

1. **Tempo de Geração vs. Escala (Tamanho da Malha):** Para malhas pequenas de 100×100 vértices, o modo Sequencial obteve média de 25,41 ms contra 55,32 ms do modo Paralelo. Em malhas maiores de 1000×1000 vértices, o tempo sequencial elevou-se para 2.295,97 ms, enquanto o paralelo registrou 4.859,25 ms. O speedup individual médio é medido em aproximadamente 0,46x e 0,47x, evidenciando que o processamento paralelo de uma única tarefa é cerca de duas vezes mais lento que o sequencial.
2. **Tempo de Geração vs. Oitavas (Complexidade do Ruído):** Com 1 oitava de ruído, a média do modo Sequencial foi de 131,01 ms contra 295,96 ms do modo Paralelo (speedup de 0,44x). Com a complexidade máxima de 10 oitavas, as médias foram de 605,42 ms e 1.192,29 ms, respectivamente, resultando em um speedup de 0,51x.

Table 1: Benchmark do tempo de geração de malha em função da escala.

Escala	Seq. Médio (ms)	Par. Médio (ms)	Speedup
100x100	25,41 ± 0,05	55,32 ± 2,11	0,46x
200x200	98,52 ± 0,16	216,95 ± 8,85	0,45x
300x300	218,54 ± 0,18	477,07 ± 20,19	0,46x
400x400	380,11 ± 0,28	808,81 ± 32,80	0,47x
500x500	585,75 ± 0,33	1241,59 ± 51,58	0,47x
600x600	842,92 ± 0,41	1792,47 ± 75,83	0,47x
700x700	1132,59 ± 0,69	2396,33 ± 100,96	0,47x
800x800	1483,53 ± 0,87	3182,91 ± 137,72	0,47x
900x900	1871,30 ± 1,08	3967,01 ± 168,51	0,47x
1000x1000	2295,97 ± 0,78	4859,25 ± 207,72	0,47x

Table 2: Benchmark do tempo de geração de malha em função das oitavas de ruído.

Oitavas	Seq. Médio (ms)	Par. Médio (ms)	Speedup
1	131,01 ± 0,28	295,96 ± 10,75	0,44x
2	176,00 ± 0,12	387,60 ± 12,80	0,45x
3	222,53 ± 0,11	496,05 ± 19,30	0,45x
4	270,30 ± 0,17	586,36 ± 21,92	0,46x
5	320,57 ± 0,23	688,34 ± 27,81	0,47x
6	374,70 ± 0,27	808,35 ± 35,60	0,46x
7	433,60 ± 0,32	895,33 ± 40,17	0,48x
8	491,28 ± 0,34	995,85 ± 48,09	0,49x
9	548,77 ± 0,33	1089,94 ± 52,81	0,50x
10	605,42 ± 0,39	1192,29 ± 57,34	0,51x

A aparente contradição da versão paralela ser mais lenta para processar uma única malha (latência da tarefa) é explicada ao analisar o tempo total necessário para processar o lote completo de testes (vazão ou *throughput*). Enquanto o lote completo no modo Sequencial levou 250,32 segundos para ser concluído, o modo Paralelo finalizou todo o trabalho em apenas 46,69 segundos — representando um **speedup global de 5,35x**.

Essa diferença de comportamento entre a latência unitária e a vazão global deve-se ao fato do TaskMaster distribuir as diferentes repetições do benchmark concorrentemente entre os núcleos físicos da CPU. Embora cada thread sofra com o *overhead* de organização e sincronização, a execução paralela de múltiplas tarefas independentes maximiza o uso do processador.

Fisicamente, a perda de desempenho individual nas execuções paralela é justificada pela disputa por recursos de memória. Os dados coletados utilizando contadores de hardware (*perf*) apontam que o modo Sequencial apresentou uma taxa de erro de cache (*cache misses*) de 30,93%, enquanto o modo Paralelo subiu para 36,48%. A execução simultânea de múltiplas threads de geração de malha força a CPU a realizar acessos frequentes à memória RAM física. Isso resulta em um aumento significativo de *cache misses*, o que explica a queda de desempenho individual. Enquanto o ganho global de vazão é evidenciado pela redução drástica do tempo total necessário para processar o lote completo de malhas.

4.2.1 Variabilidade

A análise de variabilidade dos tempos obtidos em cada execução revela grandes diferenças no comportamento de ambos os modos. No modo Sequencial, a dispersão dos dados é quase inexistente, com desvio padrão de apenas 3,99 ms no cenário de escala de 1000 × 1000 vértices. Visualmente, isso se traduz em boxplots extremamente achatados, indicando alta previsibilidade. Como a execução ocorre de forma linear e ininterrupta em um único núcleo (neste caso o núcleo 2, para evitar interrupções de sistema), os tempos permanecem constantes sob as mesmas condições.

Por outro lado, o modo Paralelo exibe uma dispersão alta, com o desvio padrão atingindo 1.059,81 ms para o mesmo tamanho de malha de 1000 × 1000. Esse comportamento se reflete em caixas amplas nos boxplots, como ilustrado em Figure 2 e Figure 3. Fisicamente, essa instabilidade é causada pela concorrência com o controle do sistema operacional. O agendamento dinâmico de threads do TaskMaster entre diferentes núcleos da CPU introduz latências causadas por concorrência de barramento de memória,

trocas de contexto (*context switching*) e atrasos na aquisição de locks da sincronização. Consequentemente, o tempo de conclusão de cada simulação individual difere de acordo com o estado da CPU e do escalonador do SO.

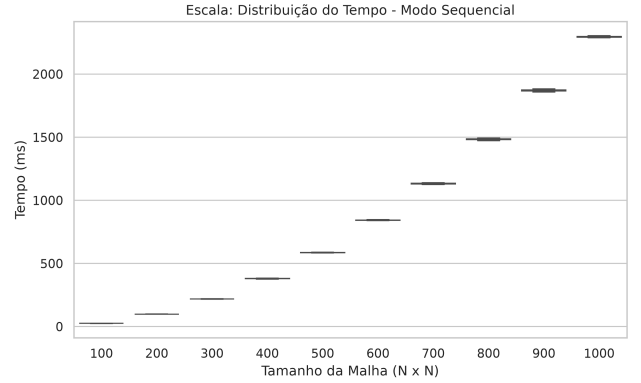


Figure 2: Dispersão do tempo de geração por escala no modo Sequencial.

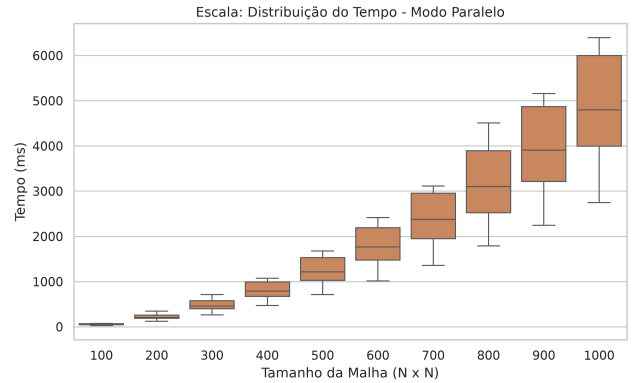


Figure 3: Dispersão do tempo de geração por escala no modo Paralelo.

4.2.2 Análise Estatística

Para avaliar de forma cientificamente se as diferenças observadas entre os tempos médios de geração dos modos Sequencial e Paralelo são estatisticamente significativas, realizou-se uma análise baseada em testes de hipóteses:

- **Hipótese Nula (H_0):** Não há diferença significativa nas médias dos tempos de geração entre os modos Sequencial e Paralelo para uma mesma configuração de parâmetros.
- **Hipótese Alternativa (H_1):** Há uma diferença estatisticamente significativa entre as médias de tempo de geração dos modos.

Primeiramente, aplicou-se a Análise de Variância de Duas Vias (*Two-Way ANOVA*) para avaliar a influência isolada do modo de execução (Sequencial ou Paralelo), do valor do parâmetro (Escala ou Oitavas) e sua interação [6]:

- **Experimento de Escala:** Revelou efeitos muito significativos para todos os fatores. O fator modo de execução obteve p-valor < 0,001, o fator Escala registrou p-valor < 0,001 e a interação entre ambos alcançou p-valor < 0,001.
- **Experimento de Oitavas:** Também demonstrou significância estatística. O fator Modo registrou p-valor < 0,001, o fator Oitavas registrou p-valor < 0,001, enquanto o fator de interação obteve p-valor < 0,001.

A forte significância estatística da interação (p-valor < 0,001) em ambos os experimentos aponta que a diferença de desempenho entre os modos Sequencial e Paralelo depende diretamente do

nível do parâmetro avaliado. Para isolar essas diferenças específicas em cada nível, aplicou-se o teste pós-hoc de Tukey [6].

No experimento de Escala, constatou-se que para grids pequenos de 100×100 (p-valor = 1,00) e 200×200 (p-valor = 0,79), a diferença entre os modos não é estatisticamente significativa. Nesses cenários, os dois algoritmos comportam-se de forma equivalente. Porém, a partir da escala 300×300 até a escala máxima de 1000×1000 , a hipótese nula H_0 foi consistentemente rejeitada (p-valor < 0,05), provando estatisticamente o atraso provocado pelo processamento paralelo de malhas individuais.

No experimento de Oitavas, a diferença foi significativa em todas as oitavas (de 1 a 10), com a rejeição da hipótese nula ocorrendo de forma estável (p-valor < 0,001) para todos os cenários de complexidade.

4.3 Desempenho no motor gráfico

Com a integração do TaskMaster ao motor gráfico, o impacto do processamento paralelo na fluidez da renderização foi avaliado através da métrica de FPS (Frames Per Second). Os resultados indicam que, mesmo com o aumento da latência individual para a geração de cada malha, a arquitetura concorrente permite que a thread principal mantenha uma taxa de quadros estável, evitando quedas bruscas de FPS e travamentos visíveis.

Nos testes a taxa de quadros por segundo foi limitada a 60 FPS para garantir uma experiência fluida. O modo Sequencial, ao bloquear a thread principal durante a geração da malha, resultou em quedas significativas de FPS, especialmente em configurações de alta complexidade (grids maiores e mais oitavas). Em contraste, o modo Paralelo conseguiu manter a taxa de quadros estável em 60 FPS, mesmo com o aumento da latência de geração, demonstrando a eficácia da arquitetura concorrente em isolar a thread de renderização das tarefas pesadas de processamento.

Ao analisar o comportamento do gerador de malhas integrado ao laço principal de renderização da engine, observam-se padrões de desempenho diferentes quanto à responsividade e à latência de geração:

- Desempenho por Escala (Tamanho da Malha):** Para malhas de 100×100 vértices, a geração em modo Sequencial ocupa a thread principal por 25,6 ms, limitando a renderização a 39 FPS. O modo Paralelo necessita de 49,8 ms para gerar a malha, mas a engine mantém-se estável a 60 FPS. Com grids de 200×200 , o tempo sequencial sobe para 99,0 ms e a taxa cai para 10 FPS, gerando engasgos visíveis. O paralelo consome 179,9 ms, mas sustenta a renderização, ainda, a 60 FPS. Na escala máxima de 1000×1000 , o modo Sequencial bloqueia a thread de renderização por 2,29 segundos resultando em 0,44 FPS, enquanto o Paralelo consome 3,89 segundos de computação secundária mantendo a fluidez estável a 60 FPS.
- Desempenho por Oitavas (Complexidade do Ruído):** Com 1 oitava de ruído, a latência de 131,0 ms no modo Sequencial reduz o jogo a 7,61 FPS, contra 60 FPS no modo Paralelo (computação de 235,3 ms). Sob complexidade máxima de 10 oitavas, o modo Sequencial desaba para 1,65 FPS (605,4 ms de latência), enquanto o Paralelo sustenta os mesmos 60 FPS, demandando 1.042,8 ms de tempo de CPU em segundo plano.

Os dados obtidos na engine gráfica para as variações de escala e de oitavas de ruído estão consolidados na Table 3 e na Table 4, respectivamente.

Table 3: Tempo de processamento e taxa de quadros (FPS) em função da escala na engine gráfica.

Escala	Seq. Tempo (ms)	Seq. FPS	Par. Tempo (ms)	Par. FPS
100x100	25,6	39	49,8	60
200x200	99,0	10	179,9	60
300x300	218,2	4,57	411,5	60
400x400	380,0	2,63	703,3	60
500x500	585,3	1,71	1071,0	60
600x600	842,4	1,19	1511,0	60
700x700	1131,7	0,88	2045,3	60
800x800	1481,8	0,67	2658,7	60
900x900	1871,7	0,53	3333,7	60
1000x1000	2295,0	0,44	3897,5	60

Table 4: Tempo de processamento e taxa de quadros (FPS) em função das oitavas de ruído na engine gráfica.

Oitavas	Seq. Tempo (ms)	Seq. FPS	Par. Tempo (ms)	Par. FPS
1	131,0	7,61	235,3	60
2	176,3	5,67	308,5	60
3	222,5	4,49	402,6	60
4	270,4	3,70	511,6	60
5	320,2	3,12	581,9	60
6	374,1	2,67	689,4	60
7	433,1	2,31	804,4	60
8	490,7	2,04	907,6	60
9	548,0	1,82	1010,8	60
10	605,4	1,65	1042,8	60

A dispersão do FPS obtido durante a simulação por escala pode ser observada na Figure 4 e na Figure 5. Os gráficos contrastam a instabilidade e a perda acentuada de FPS do modo sequencial sob cargas altas com a estabilidade do modo paralelo no limite físico do motor gráfico.

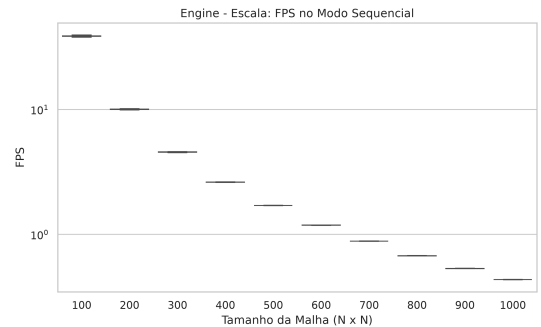


Figure 4: Dispersão da taxa de quadros (FPS) por escala no modo Sequencial na engine gráfica.

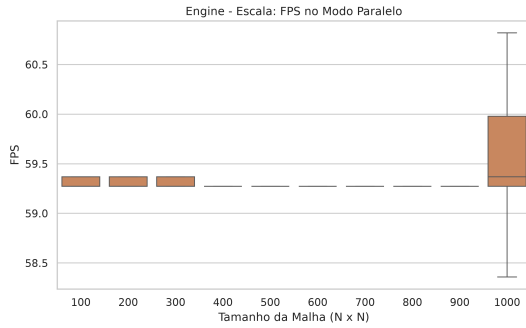


Figure 5: Dispersão da taxa de quadros (FPS) por escala no modo Paralelo na engine gráfica.

Cabe notar uma particularidade de visualização na Figure 5: embora os limites dos diagramas de caixa (*whiskers*) e do corpo da caixa aparentem cobrir uma grande área da escala vertical, isso é um artefato visual decorrente do ajuste automático de escala do eixo vertical no *software* de plotagem. Como a variação real do FPS no modo paralelo é quase nula (na ordem de 10^{-1} a 10^{-2} FPS), o eixo vertical foi ampliado em um intervalo microscópico.

Essa variação quase inexistente de FPS para a maioria das escalas ocorre porque a transferência de malhas pequenas para a GPU consome tempo desprezível da thread principal. Contudo, na escala máxima de 1000×1000 vértices, a malha possui cerca de um milhão de vértices. O envio desse grande volume de dados de vértices é processado obrigatoriamente pela thread principal de renderização. O *upload* desse buffer de dados no momento em que a malha fica pronta consome alguns milissegundos do tempo limite do quadro, explicando o desvio padrão de 0,56 FPS e as oscilações entre 57,8 e 60,8 FPS.

4.3.1 O Paradoxo da Responsividade

Embora o modo paralelo resulte em uma maior latência absoluta para concluir uma única tarefa (como visto anteriormente), a delegação desse processamento a threads secundárias pelo TaskMaster impede o bloqueio do laço de renderização principal. Assim, para aplicações gráficas interativas em tempo real, a estabilidade e a responsividade mostram-se mais importantes que o tempo bruto de execução do algoritmo de forma isolada.

4.3.2 Métricas de Cache na Engine

Os contadores físicos de CPU coletados via *perf* durante a execução junto ao motor gráfico corroboram as conclusões do benchmark isolado a respeito da disputa por recursos de memória. No modo Sequencial, a taxa de *cache misses* registrou 32,77%. Sob a execução do modo Paralelo, essa taxa subiu para 41,54%. Essa diferença reforça a explicação de que o processamento paralelo de múltiplas tarefas simultâneas aumenta significativamente a pressão sobre o subsistema de memória, resultando em um aumento substancial de *cache misses*. No entanto, mesmo com essa penalidade de desempenho individual, a arquitetura concorrente do TaskMaster permite que a aplicação mantenha uma experiência fluida e responsiva, além de alavancar o desempenho de gerações em massa.

4.3.3 Análise Estatística

Para consolidar as conclusões observadas no motor gráfico, aplicou-se a Análise de Variância (ANOVA) de duas vias sobre a taxa de quadros e o tempo de geração. A análise confirmou que o modo de execução possui efeito altamente significativo no tempo de processamento ($p\text{-valor} < 0,001$). O fator modo de execução

(Sequencial ou Paralelo) também apresentou impacto estatístico massivo especificamente sobre a taxa de quadros $p\text{-valor} < 0,001$.

Por fim, o teste pós-hoc de Tukey corroborou que a melhoria de FPS obtida pela arquitetura concorrente é estatisticamente significativa em todas as escalas e oitavas avaliadas com $p\text{-valor} < 0,001$, validando cientificamente a eficácia da solução paralela.

5 CONCLUSÃO

Este trabalho apresentou uma arquitetura concorrente assíncrona baseada no padrão *Task Scheduler*, implementada em C++20 através da classe *TaskMaster*, para solucionar o gargalo de processamento na geração procedural de terrenos e extração de malhas tridimensionais integradas a motores gráficos baseados em OpenGL. O foco principal foi garantir a responsividade e a estabilidade da taxa de quadros (FPS) ao delegar tarefas intensivas para threads secundárias.

Os resultados experimentais revelaram um comportamento interessante. Embora a latência individual para a geração de uma única malha tenha aumentado no modo paralelo devido à disputa por recursos de memória e a um incremento nas falhas de cache (*cache misses* de 30,93% para 36,48%), o ganho global de vazão foi massivo, com um *speedup* global de 5,35x no processamento em lote. No motor gráfico, enquanto a abordagem sequencial inviabilizou a renderização ao bloquear a *main thread* por mais de 2 segundos em malhas densas (atingindo 0,44 FPS), a arquitetura proposta sustentou de forma consistente o limite físico de 60 FPS da aplicação. As análises estatísticas por ANOVA de duas vias e teste pós-hoc de Tukey comprovaram com alta significância ($p\text{-valor} < 0,001$) a eficácia da paralelização.

Do ponto de vista de engenharia de software, o uso dos recursos modernos do C++20, como `std::jthread`, `std::stop_token` e ponteiros inteligentes, mostrou-se fundamental. Estes mecanismos simplificaram o gerenciamento do ciclo de vida das threads e garantiram a segurança de memória contra condições de corrida e travamentos, demonstrando que a concorrência moderna em C++ reduz substancialmente a complexidade do código e o risco de vazamentos de recursos.

Como trabalhos futuros, sugere-se a investigação de técnicas de transferência de dados mais eficientes para a GPU para mitigar o *overhead* observado no envio de buffers muito grandes da *main thread*, utilizando instanceamento (*Instancing*). Adicionalmente, propõe-se explorar a migração dos algoritmos de geração de ruído e extração de geometria diretamente para a GPU, avaliando o ganho de desempenho em comparação com a arquitetura concorrente baseada em CPU proposta neste artigo.

REFERÊNCIAS

- [1] Wilton de O. Bussab and Pedro A. Morettin. 2017. *Estatística Básica* (9ª ed.). Saraiva, São Paulo.
- [2] Raj Jain. 1991. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, New York.
- [3] Ricardo de la Rocha Ladeira. 2024. Aula 5 - Sincronização.
- [4] Ricardo de la Rocha Ladeira. 2024. Aula 4 - Threads.
- [5] David J. Lilja. 2000. *Measuring Computer Performance: A Practitioner's Guide*. Cambridge University Press, Cambridge.
- [6] Douglas C. Montgomery. 2017. *Design and Analysis of Experiments* (9th ed.). John Wiley & Sons, Hoboken, NJ.
- [7] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. 2009. Producing wrong data without doing anything obviously wrong!. In *Proceedings of the 14th In-*

- ternational Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*, 2009. ACM, 265–276. <https://doi.org/10.1145/1508244.1508275>
- [8] Ken Perlin. 1985. An Image Synthesizer. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, 1985. Association for Computing Machinery, New York, NY, USA, 287–296. <https://doi.org/10.1145/325334.325247>
 - [9] Joey de Vries. 2014. Learn OpenGL: Enjoy learning modern OpenGL. Retrieved April 18, 2026 from <https://learnopengl.com/>
 - [10] Anthony Williams. 2019. *C++ Concurrency in Action* (2nd ed.). Manning Publications.